

Graph, Tree & DFS Masterclass Notes

Aapki Memory & Python Concepts ke Sath Jode Gaye Ultimate Bachchon Jaise Notes

1. Memory Link & Quick Context (Aapke Sath Connection)

Aapne abhi-abhi Python mein loops (jaise `for i in range(5):`) aur repetition operator (*) ke baare mein seekha hai [cite: BgIVVxERhig]. Inhi chote-chote python tools ka use karke hum aaj **Graph Traversal (DFS)** ka poora software logic build karenge [cite: BgIVVxERhig]!

2. Theory vs Graph aur Tree vs Graph ka Farq

Sabse pehle yeh clear karte hain ki **Theory** aur **Graph** mein kya farq hai, aur phir DSA ke andar **Tree** aur **Graph** kaise alag hote hain [cite: BgIVVxERhig]!

Concept	Kya Matlab Hai?	Asan Bhasha / Example
Theory vs Graph	Theory rules/concepts samjhati hai; Graph data ke beech ka relationship visual form mein dikhata hai.	Theory = Newton ke laws ki definition; Graph = Speed aur Time ka line chart.
Tree vs Graph	Tree ek hierarchical structure hai (no cycles); Graph ek network hai jismein cycles/loops ho sakte hain [cite: BgIVVxERhig].	Tree = Folder structure (Parent-Child); Graph = Social media friends network ya Google Maps [cite: BgIVVxERhig].

Colorful Diagram Concept: Tree vs Graph

Tree: [Root: 1] → [Node: 2] → [Node: 3] (*Wapas 1 par nahi aa sakte - Acyclic*)

Graph: [1] ⇌ [2] ⇌ [3] ∪ [1] (*Gol-gol ghum sakte hain - Cyclic*) [cite: BgIVVxERhig]

3. DFS (Depth First Search) - Bhool-Bhulaiyaa Wali Kahani

Sochiye aap ek bhool-bhulaiyaa (Maze) mein fase hain. Aapko exit chahiye [cite: BgIVVxERhig, 11:54]. Aap kya karoge? Ek rasta pakad kar tab tak deep (gehrayi) mein jaoge jab tak rasta band (Dead End) na ho jaye [cite: BgIVVxERhig, 07:20]!

DFS ke 3 Golden Steps:

1. **Go Deep First:** Jab tak aage rasta khula hai, aage badhte jao, piche mat dekho [cite: BgIVVxERhig, 07:20].
2. **Visited Array (Yaad Rakhne Wala Box):** Jo node visit kar liya, use mark kar do taaki baar-baar wahan na ghumo [cite: BgIVVxERhig, 09:38].
3. **Backtrack via Stack:** Jab dead end aa jaye, toh piche mudne (backtrack) ke liye Stack ka use hota hai [cite: BgIVVxERhig, 14:03].

4. Complete Python Code for DFS (Adjacency Matrix & Stack)

Yahan wahi repetition operator * use hota hai jo aapne abhi seekha tha (jaise [False] * self.size) [cite: BgIVVxERhig, 28:22]!

```
class Graph:
    def __init__(self, size):
        self.size = size
        # Matrix initialization using repetition operator '*'
        self.matrix = [[0] * size for _ in range(size)]

    def add_edge(self, u, v):
        self.matrix[u][v] = 1

    def dfs(self, start_node):
        # Visited tracking array
        visited = [False] * self.size
        stack = [start_node]

        while stack:
            v = stack.pop()
            if not visited[v]:
                print(v, end=" ")
                visited[v] = True

            # Check neighbors in reverse order
            for i in range(self.size - 1, -1, -1):
                if self.matrix[v][i] == 1 and not visited[i]:
                    stack.append(i)

# Testing the Graph:
g = Graph(6)
g.add_edge(0, 1)
g.add_edge(0, 2)
g.add_edge(2, 3)
g.add_edge(2, 4)
g.add_edge(3, 5)
g.add_edge(4, 5)

print("DFS Traversal Output:")
g.dfs(0)
```

Important Exam / Interview Point: Tree mein traversal ka output duniya mein sabka **unique** hota hai, lekin Graph mein alag-alag logo ka output alag ho sakta hai kyunki starting node ya neighbor order alag ho sakta hai [cite: BgIVVxERhig, 03:54]!

5. Minimum Spanning Tree (MST) Kya Hota Hai?

Jab aap Graph ke andar DFS chalate waqt saari cyclic (gol-gol ghumane wali) edges ko hata dete ho, toh jo saaf-suthra tree bachta hai, use hi **Minimum Spanning Tree (MST)** kehte hain [cite: BgIVVxERhig, 25:12]!